HANDBOOK

Web Developer

Grade XII – IT-ITeS Vocational Education

MySQL • SQL • PHP • Workplace Safety

A structured textbook for database development, server-side programming and responsible workplaces.

Developed with reference to the topics in the supplied Grade XII draft study material.

Contents

1 Database Management Concepts **1**
1.1 Data, databases and DBMS . . . 1
1.2 Tables, records, fields and keys . . . 1
1.3 Relational and other database models . . . 2
1.4 File systems and database systems . . . 2
1.5 Normalisation and data quality . . . 2
1.6 Transactions and ACID . . . 2

2 Structured Query Language and MySQL **4**
2.1 Main categories of SQL commands . . . 4
2.2 Creating a database and table . . . 4
2.3 Retrieving and filtering data . . . 5
2.4 Adding, updating and deleting records . . . 5
2.5 Functions, grouping and aggregation . . . 5
2.6 Joins and relationships . . . 5
2.7 Installation and safe database practice . . . 6

3 SQL Functions and Data Handling **7**
3.1 String functions . . . 7
3.2 Numeric functions . . . 7
3.3 Date and time functions . . . 7
3.4 Aggregate functions and NULL values . . . 7
3.5 Permissions and transactions . . . 7

4 Introduction to PHP and Server-Side Scripting **9**
4.1 How a PHP request works . . . 9
4.2 Running a first PHP script . . . 9
4.3 Variables, types and operators . . . 10
4.4 Conditions and loops . . . 10
4.5 Forms and request data . . . 10

5 PHP Arrays, Strings and Functions **11**
5.1 Arrays . . . 11
5.2 Working with strings . . . 11
5.3 Functions and parameters . . . 11
5.4 Server variables and HTTP headers . . . 12
5.5 Error handling and maintainability . . . 12

6 PHP and MySQL Database Applications **13**
6.1 Connection choices . . . 13
6.2 Prepared statements . . . 13
6.3 Create, read, update and delete . . . 14
6.4 SQL injection . . . 14

6.5 Output encoding and sessions 14
6.6 A responsible development checklist 14

7 Productivity, Quality and Professional Work Habits 15
7.1 Planning and prioritising 15
7.2 Quality and continuous improvement 15
7.3 Communication and teamwork 15
7.4 Managing digital work responsibly 15

8 Health and Safety Procedures in the Workplace 17
8.1 Hazard and risk 17
8.2 Hierarchy of controls 17
8.3 Computer-work ergonomics 18
8.4 Electrical safety and emergency response 18
8.5 Incident reporting 18

9 Waste Management, Recycling and Resource Conservation 19
9.1 Identify and separate waste 19
9.2 The waste hierarchy 19
9.3 Electronic waste 20
9.4 Conserving energy and water 20
9.5 Planning a small waste programme 20

10 Diversity, Equality and Inclusive Workplaces 21
10.1 Diversity and equality 21
10.2 Everyday inclusive behaviour 21
10.3 Discrimination and harassment 21
10.4 Gender equality and work-life responsibilities 21
10.5 Building an inclusive team 21

11 Accessibility and Policies for Persons with Disabilities 22
11.1 Recognising different access needs 22
11.2 Digital accessibility 22
11.3 Physical and workplace adjustments 22
11.4 Respectful support and privacy 22

12 Integrated Case Study and Revision 24
12.1 Case study: a training-centre portal 24
12.2 Plan the database 24
12.3 Plan the application 24
12.4 Plan quality and workplace responsibility 24
12.5 Revision questions 24
12.6 Answer-planning guide 25

Chapter 1

Database Management Concepts

A database becomes useful when information can be stored consistently, found quickly and updated without creating confusion. A school may need student records, a shop may maintain stock and sales, and a web application may keep account details and orders. A database management system (DBMS) provides the tools needed to organise this information and control how people and applications use it.

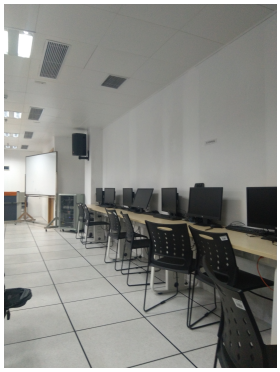


Figure 1.1. A computer room with server equipment illustrates infrastructure that supports database systems. Photograph: StellaAnnie, Wikimedia Commons, CC BY-SA 4.0.

1.1 Data, databases and DBMS

Data is a recorded value such as a name, date, price or attendance status. A database is an organised collection of related data. A DBMS is software that creates, stores, retrieves and protects that data. MySQL is a widely used relational DBMS; other examples include PostgreSQL, Oracle Database and Microsoft SQL Server.

A database system supports more than storage. It can enforce rules, control access, manage simultaneous users, support backup and recovery, and help applications retrieve only the information they need. These controls reduce the risk of inconsistent or lost records.

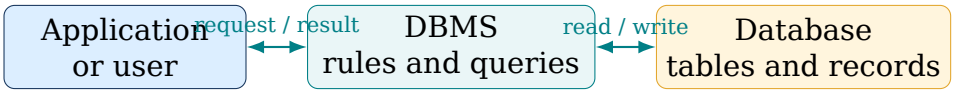


Figure 1.2. The DBMS mediates between applications and stored data.

1.2 Tables, records, fields and keys

In a relational database, a table stores one kind of entity. Rows represent records; columns represent attributes. A student table might contain student ID, name, class and date of admission. Each column has a data type, and each row should be identifiable in a reliable way.

A **primary key** uniquely identifies each row and cannot be missing. A **foreign key** links a row to a matching key in another table. A **unique constraint** prevents duplicate values in a column or column group. A **NOT NULL constraint** requires a value,

while a **CHECK constraint** can enforce a condition supported by the database engine.

1.3 Relational and other database models

Relational databases organise data in related tables and use SQL to work with it. NoSQL databases use other models, including document, key-value, column-family and graph structures. The appropriate model depends on the shape of the data, consistency needs, access patterns and operational requirements; no single model is best for every application.

1.4 File systems and database systems

A file system stores files and folders. It is appropriate for documents, images and many ordinary files. A DBMS adds structured queries, constraints, relationships and transaction handling that are valuable when many records must be managed together. File storage and databases often work side by side in one application.

Aspect	File-based approach	Database approach
Structure	Files and folders; format depends on the application	Tables, columns, types and relationships
Searching	Often depends on filenames or application-specific logic	SQL queries can filter, sort and join records
Integrity	Rules are often implemented separately in code	Constraints can be enforced centrally
Concurrent changes	Applications must coordinate carefully	DBMS provides transaction and concurrency controls
Security	File permissions and application controls	Database accounts, privileges and application controls
Recovery	File backups and application procedures	Backup, logging and database recovery features

1.5 Normalisation and data quality

Normalisation is a way of organising tables to reduce unnecessary repetition and update anomalies. For example, storing a department's address in every employee row means an address change must be repeated in many places. A separate department table allows employees to refer to one department record. Good normalisation begins with clear entities, keys and relationships; it should also reflect actual reporting and performance needs.

1.6 Transactions and ACID

A transaction groups operations that should succeed or fail as a unit. ACID summarises four important properties: Atomicity (all or nothing), Consistency (valid rules remain satisfied), Isolation (concurrent work is controlled), and Durability (committed changes persist according to the system's guarantees). Transactions matter in banking, ordering, stock control and any workflow where partial updates would create a misleading result.

Review Questions

1. Define DBMS and give two examples.

2. Distinguish a row from a column.
3. Explain primary and foreign keys.
4. State two advantages and two limitations of a file-based approach.
5. What does each ACID property protect?

Chapter 2

Structured Query Language and MySQL

SQL is used to define database structures, retrieve records, change data and manage access. Although database products differ in some details, the basic ideas of tables, filtering, sorting, joins and constraints are widely shared.



Figure 2.1. A code screen illustrates the syntax used to write and review database queries. Photograph: Lorenzo Cafaro, Wikimedia Commons, CC0 1.0.

2.1 Main categories of SQL commands

- **DDL (Data Definition Language):** CREATE, ALTER and DROP define or change database objects.
- **DML (Data Manipulation Language):** INSERT, UPDATE and DELETE change records.
- **DQL (Data Query Language):** SELECT retrieves information; this label is often used for query statements.
- **DCL (Data Control Language):** GRANT and REVOKE manage privileges.
- **TCL (Transaction Control Language):** COMMIT and ROLLBACK manage transaction outcomes.

2.2 Creating a database and table

The following examples use MySQL-style syntax. Run them in a practice database, not against real records.

```
CREATE DATABASE training_centre;
USE training_centre;

CREATE TABLE students (
  student_id INT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  course_name VARCHAR(80),
  marks DECIMAL(5,2) CHECK (marks >= 0 AND marks <= 100)
);
```

Choose data types that fit the values. INT is suitable for whole numbers; VARCHAR stores variable-length text; DATE stores a calendar date; DECIMAL is appropriate

for exact decimal quantities such as money. Avoid storing every value as text when calculations, sorting or validation matter.

2.3 Retrieving and filtering data

SELECT names the columns to return. WHERE filters rows, ORDER BY sorts the result, and LIMIT can restrict the number of returned rows in MySQL.

```
SELECT student_id, full_name, marks
FROM students
WHERE marks >= 60
ORDER BY marks DESC
LIMIT 10;
```

Use explicit column names rather than relying on SELECT * in long-lived application queries. Explicit selections make the expected output easier to review and reduce accidental exposure when a table gains new columns.

2.4 Adding, updating and deleting records

```
INSERT INTO students (student_id, full_name, course_name, marks)
VALUES (101, 'Ananya Rao', 'Web Development', 86.50);

UPDATE students
SET marks = 88.00
WHERE student_id = 101;

DELETE FROM students
WHERE student_id = 101;
```

UPDATE and DELETE should be used with particular care. Before running a change, check the WHERE condition with a SELECT query and confirm that it identifies exactly the intended records. In real systems, use appropriate permissions, backups and transactions.

2.5 Functions, grouping and aggregation

SQL functions can transform values or summarise a set of records. String functions include UPPER, LOWER, CONCAT and LENGTH; numeric functions include ROUND, CEIL, FLOOR and ABS; date functions depend on the DBMS. Aggregate functions such as COUNT, SUM, AVG, MIN and MAX calculate a summary across rows.

```
SELECT course_name,
       COUNT(*) AS learner_count,
       ROUND(AVG(marks), 2) AS average_marks
FROM students
GROUP BY course_name
ORDER BY average_marks DESC;
```

GROUP BY creates groups for aggregation. HAVING filters groups after aggregation, while WHERE filters rows before grouping. NULL values require care: most aggregate functions ignore NULL inputs, and comparisons with NULL use IS NULL or IS NOT NULL rather than ordinary equality.

2.6 Joins and relationships

A JOIN combines rows from related tables. An INNER JOIN returns matching rows. A LEFT JOIN retains every row from the left table and includes matching data from

the right where available. Joins are easier to reason about when key columns are named clearly and their relationships are documented.

```
SELECT s.full_name, c.course_title
FROM students AS s
INNER JOIN courses AS c
  ON s.course_id = c.course_id;
```

2.7 Installation and safe database practice

For learning, use the official MySQL Community Edition download and follow current installation instructions for your operating system. A typical setup may include MySQL Server and a client such as MySQL Workbench. Create a dedicated practice account, use a strong password, keep the server local while learning, and do not expose the database port to the public internet without a carefully designed security plan.

Review Questions

1. Compare DDL and DML.
2. What is the purpose of a primary key?
3. Write a query to sort marks from highest to lowest.
4. Explain the difference between WHERE and HAVING.
5. Compare INNER JOIN and LEFT JOIN.
6. Why should UPDATE and DELETE statements be checked before execution?

Chapter 3

SQL Functions and Data Handling

Functions help queries express common calculations without requiring every operation to be performed manually outside the database. A function accepts a value or expression and returns a result. Exact names and behaviour can differ across database products, so check the documentation for the system in use.

3.1 String functions

UPPER and LOWER change letter case, CONCAT joins text, LENGTH reports the length according to the DBMS's rules, and TRIM removes surrounding whitespace. SUBSTRING returns part of a string. These operations are useful for formatting output, preparing search values and producing reports. They should not be used as a substitute for storing clean, well-validated data.

```
SELECT UPPER(full_name) AS display_name,
       LENGTH(full_name) AS name_length
FROM students;
```

3.2 Numeric functions

ABS returns a non-negative magnitude, ROUND rounds a number to a specified precision, and CEIL/FLOOR round upward/downward to integer boundaries. Check the effect of rounding before using it in financial or assessment calculations; the displayed value and stored value are not always the same.

3.3 Date and time functions

Date functions can extract parts of a date, compare dates, format a value or calculate intervals. MySQL provides functions such as CURDATE(), NOW(), YEAR() and `DATE_FORMAT()`. Other SQL systems may use different names or arguments. Store dates using date/time types when possible, rather than ambiguous strings such as 03/04/25.

3.4 Aggregate functions and NULL values

COUNT counts rows or non-NULL values depending on its argument; SUM totals numeric values; AVG computes a mean; MIN and MAX return the smallest and largest values. NULL means that a value is unknown or absent, not zero and not an empty string. Queries should handle missing values intentionally.

```
SELECT COUNT(*) AS total_records,
       COUNT(marks) AS records_with_marks,
       AVG(marks) AS mean_marks,
       MIN(marks) AS lowest_marks,
       MAX(marks) AS highest_marks
FROM students;
```

3.5 Permissions and transactions

GRANT assigns permitted actions to a database account; REVOKE removes permissions. Follow least privilege: give each account only the access it needs. COMMIT

confirms a transaction and ROLLBACK abandons uncommitted changes. Transaction support and behaviour depend on the storage engine and the operations being performed.

Review Questions

1. Give examples of string and numeric functions.
2. Compare COUNT(*) with COUNT(column).
3. Why should dates be stored using date types?
4. What is NULL?
5. Explain least privilege in database access.

Chapter 4

Introduction to PHP and Server-Side Scripting

PHP is a server-side scripting language often used to create dynamic web pages and web services. The server executes PHP code and sends a response to the browser. Visitors normally receive the resulting HTML, not the server’s PHP source. PHP can process form input, apply application rules, communicate with databases and generate a response.



Figure 4.1. Server-side scripts are written and tested in a programming environment. Photograph: Martin Vorel, Wikimedia Commons, CC BY-SA 4.0.

4.1 How a PHP request works



Figure 4.2. A typical request-response path for a server-side application.

4.2 Running a first PHP script

Use a supported PHP installation or a local development bundle, and follow the current official installation instructions. Save the following as `hello.php` in the configured project directory and open it through the local development server.

```
<?php
$course = "Web Development";
echo "<h1>Welcome to " . htmlspecialchars($course, ENT_QUOTES, 'UTF-8') . "
    </h1>";
?>
```

PHP variables begin with a dollar sign. Statements commonly end with a semicolon. The `echo` construct outputs content. When displaying user-controlled text inside HTML, escape it for the correct output context; escaping is a security control, not just a formatting choice.

4.3 Variables, types and operators

Variables hold values such as strings, integers, floating-point numbers, booleans, arrays and objects. PHP is dynamically typed, but developers should still validate input and make assumptions clear. Arithmetic operators calculate values; comparison operators compare values; logical operators combine conditions. Use strict comparisons where appropriate and avoid depending on surprising type conversions.

4.4 Conditions and loops

```php
$marks = 74;
if ($marks >= 80) {
    echo "Excellent";
} elseif ($marks >= 50) {
    echo "Pass";
} else {
    echo "Needs more practice";
}
```

Conditions allow code to choose between actions. Loops repeat work while a condition holds or for each item in a collection. Every loop needs a sensible stopping condition so that it does not run forever.

4.5 Forms and request data

HTML forms commonly submit data with GET or POST. GET data is typically included in the URL and is suitable for retrieval requests; POST sends data in the request body and is often used for changes. POST does not automatically make data private or safe: use HTTPS, validation, access control and appropriate request protections.

```php
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    if ($name === '') {
        echo "Please enter a name.";
    } else {
        echo "Received a valid name.";
    }
}
?>
```

The example demonstrates basic checking only. A real form should include a suitable HTML form, a clear label, server-side validation, CSRF protection where relevant, safe output encoding and meaningful error handling.

Review Questions

1. What is server-side scripting?
2. Describe the journey of an HTTP request to a PHP server.
3. What is the difference between GET and POST?
4. Why should user-provided text be escaped before it is inserted into HTML?

Chapter 5

PHP Arrays, Strings and Functions

5.1 Arrays

An array stores several values under one variable. An indexed array uses numeric keys, an associative array uses named keys, and a multidimensional array contains arrays inside another array. Select the structure that best represents the data.

```php
$courses = ["HTML", "CSS", "JavaScript"];
$learner = [
    "name" => "Ananya",
    "course" => "Web Development",
    "completed" => true
];

foreach ($courses as $course) {
    echo htmlspecialchars($course, ENT_QUOTES, 'UTF-8') . "<br>";
}
```

Array functions can sort values, count elements, test for keys and transform data. Be mindful that some sorting functions reindex numeric keys, while others preserve key associations. Check the function's documented behaviour before depending on the original order.

5.2 Working with strings

String operations can trim extra spaces, change letter case, find a substring, split text or join pieces. When processing input, consider empty strings, unexpected encodings and very long values. A value should be checked according to its intended meaning, not merely accepted because it is a string.

5.3 Functions and parameters

A function gives a name to a reusable task. Parameters receive input, and a return value communicates a result. Small functions are easier to test, reuse and maintain than long blocks that mix unrelated responsibilities.

```php
function calculateAverage(array $scores): ?float {
    if (count($scores) === 0) {
        return null;
    }
    return array_sum($scores) / count($scores);
}

$average = calculateAverage([70, 80, 90]);
```

PHP passes values by value by default; a parameter can be explicitly passed by reference using an ampersand in its declaration. References should be used only when changing the caller's variable is part of the function's intended behaviour. Recursive functions call themselves and must have a base case to stop the recursion.

5.4 Server variables and HTTP headers

The `$_SERVER` superglobal provides request and server information, such as the request method and script path. Values should not be blindly trusted as proof of identity or permission. HTTP headers communicate metadata and response instructions. Header operations must be sent before body output, and redirects should be followed by an exit where appropriate.

5.5 Error handling and maintainability

Use clear names, consistent formatting, comments that explain intent, and functions with a focused purpose. During development, errors can be logged for diagnosis. Production systems should avoid showing stack traces, secrets or internal paths to visitors. Logs should not unnecessarily contain passwords, session tokens or sensitive personal information.

Review Questions

1. Compare indexed and associative arrays.
2. What is the purpose of a foreach loop?
3. Explain function parameters and return values.
4. What is a base case in recursion?
5. Why should internal error details not be displayed to every visitor?

Chapter 6

PHP and MySQL Database Applications

A web application often needs to save and retrieve information. PHP can connect to MySQL, send queries and use the returned rows to generate a response. The connection, query and output stages all need careful error handling and security controls.



Figure 6.1. Web applications depend on server systems to process requests and access stored data. Photograph: Ricardo Moctezuma López / ronK, Wikimedia Commons, CC BY-SA 2.5.

6.1 Connection choices

Modern PHP applications commonly use PDO or MySQLi. Both support prepared statements. PDO offers a consistent interface across supported database drivers; MySQLi is designed for MySQL-compatible databases. Use the interface supported by the project and keep credentials outside public source files, for example in environment configuration that is not committed to a repository.

6.2 Prepared statements

Never build a query by joining untrusted input directly into SQL text. Prepared statements separate the query structure from the supplied values and help prevent SQL injection.

```
<?php
$pdo = new PDO(
    'mysql:host=127.0.0.1;dbname=training_centre;charset=utf8mb4',
    getenv('DB_USER'),
    getenv('DB_PASSWORD'),
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

$stmt = $pdo->prepare(
    'SELECT student_id, full_name FROM students WHERE student_id = :id'
);
$stmt->execute(['id' => $studentId]);
$student = $stmt->fetch(PDO::FETCH_ASSOC);
?>
```

This is a learning pattern, not a complete production configuration. The variable `$studentId` must be validated, the credentials must be configured securely, and exceptions should be handled by the application without exposing sensitive details to the browser.

6.3 Create, read, update and delete

CRUD describes four common data operations: create a record, read records, update existing information and delete records. Each operation should verify that the user is authorised to perform it. Deleting records may require confirmation, audit logging or a retention process depending on the system's requirements.

6.4 SQL injection

SQL injection occurs when untrusted input is interpreted as part of a query's executable SQL syntax. It may expose data, bypass checks or modify records. Prepared statements are the primary defence for values; least-privilege database accounts, validation, safe error handling and security testing add further protection. Escaping alone is not a substitute for prepared statements.

6.5 Output encoding and sessions

When displaying text in HTML, use context-appropriate output encoding, such as `htmlspecialchars` for ordinary HTML text. Authentication systems should use secure session settings, regenerate session identifiers after login, enforce permissions on the server and protect sensitive actions. Hiding a button in the browser is not authorisation.

6.6 A responsible development checklist

- Use a separate database account with only the privileges needed.
- Store credentials in environment variables or a protected configuration system.
- Use prepared statements for values in SQL queries.
- Validate input on the server and encode output for its context.
- Use HTTPS for real deployments and protect sessions.
- Log useful security events without recording secrets.
- Back up data and test recovery procedures.

Review Questions

1. Name two PHP interfaces used with MySQL.
2. How do prepared statements reduce SQL injection risk?
3. What do CRUD operations mean?
4. Why is output encoding necessary?
5. Give four checks to complete before a database-backed form is released.

Chapter 7

Productivity, Quality and Professional Work Habits

Productivity is not simply completing a large number of tasks. Good work is accurate, useful and delivered with appropriate use of time and resources. In a software workplace, productivity improves when requirements are understood, tasks are prioritised, tools are maintained, communication is clear and defects are found early.



Figure 7.1. A workstation should be kept organised so equipment and materials can be used safely. Photograph: Frederick Noronha, Wikimedia Commons, CC BY-SA 4.0.

7.1 Planning and prioritising

Break large assignments into smaller tasks with clear outcomes. Estimate the effort, identify dependencies and reserve time for review. A simple task board can show work that is planned, in progress, waiting for review and complete. When priorities change, communicate the impact rather than silently missing a deadline.

7.2 Quality and continuous improvement

Quality means meeting agreed requirements reliably. Use checklists, peer review, testing and documented procedures. When an error occurs, look for the cause and improve the process instead of focusing only on blame. Useful measures might include defects found after release, rework, task completion and customer feedback; a single metric rarely tells the whole story.

7.3 Communication and teamwork

Share information in a clear and respectful way. Confirm important instructions, record decisions and raise risks early. When giving feedback, describe the observed issue, its impact and a practical next step. Protect confidential information and use approved channels for work discussions.

7.4 Managing digital work responsibly

Organise files with meaningful names, keep versions under control, back up important work and avoid saving confidential material in unapproved locations. Lock the screen when stepping away, use strong authentication and report suspicious messages or unexpected access requests.

Review Questions

1. Distinguish productivity from simply working faster.
2. Give three methods for improving quality.
3. Why should a project risk be reported early?
4. List four habits that support responsible digital work.

Chapter 8

Health and Safety Procedures in the Workplace

A safe workplace is created through planning, training, suitable equipment and shared responsibility. Hazards may be physical, electrical, ergonomic, chemical, environmental or psychosocial. Identifying a hazard early gives people a chance to remove it or reduce the risk before someone is harmed.

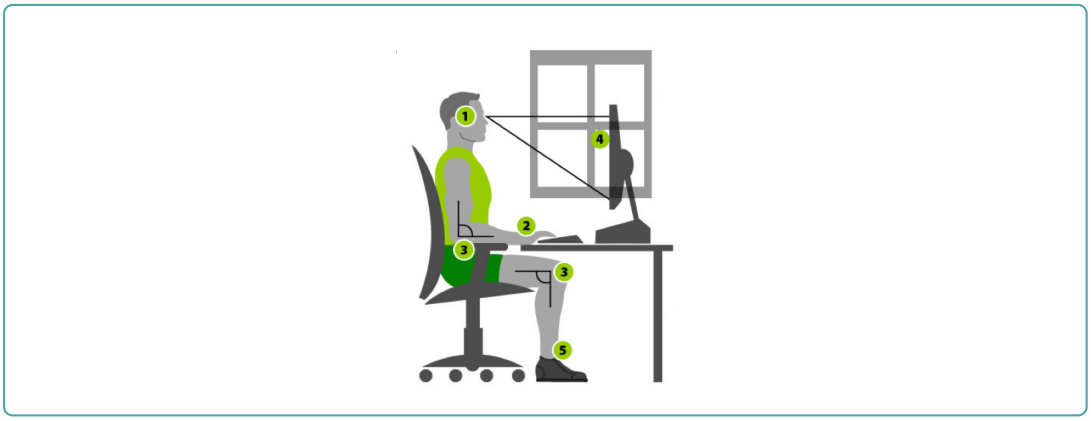


Figure 8.1. An ergonomic workstation supports comfortable computer use and reduces avoidable strain. Illustration: Marcel Kollmar and Partynia, Wikimedia Commons, CC BY-SA.

8.1 Hazard and risk

A hazard is something that can cause harm. Risk considers how likely that harm is and how serious it could be. A loose cable across a walkway is a hazard; the risk increases when people regularly walk through the area and could trip. Report hazards promptly and follow the organisation’s safety procedure.

8.2 Hierarchy of controls

Where possible, remove the hazard first. If elimination is not practical, consider substituting a safer option, using engineering controls, improving administrative procedures and using personal protective equipment where required. PPE is important but should not be the only control when more effective options are available.

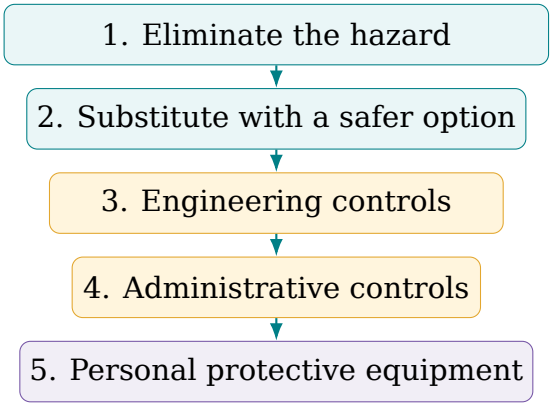


Figure 8.2. The hierarchy of controls prioritises stronger ways of reducing risk.

8.3 Computer-work ergonomics

Adjust the chair, desk and screen so the body can remain in a comfortable, neutral position. Keep frequently used items within easy reach, avoid glare and take regular short movement breaks. Repeated discomfort, headaches or eye strain should be reported and addressed rather than treated as an unavoidable part of computer work.

8.4 Electrical safety and emergency response

Do not use damaged plugs, exposed wires or overloaded extension boards. Keep liquids away from electrical equipment and never attempt repairs beyond your training or authorisation. Know emergency exits, alarms, first-aid arrangements and reporting contacts. In an emergency, follow local procedures and contact trained responders; do not put yourself at risk by attempting an unfamiliar rescue.

8.5 Incident reporting

Report injuries, near misses, unsafe conditions and equipment faults according to workplace policy. Record factual details such as location, time, what was observed and immediate controls taken. Timely reporting helps prevent recurrence and supports a safer environment for everyone.

Review Questions

1. Define hazard and risk.
2. Name the main levels in the hierarchy of controls.
3. Give five ways to improve computer-work ergonomics.
4. What should an employee do after noticing a damaged power cable?
5. Why are near misses worth reporting?

Chapter 9

Waste Management, Recycling and Resource Conservation

Workplaces use electricity, water, paper, equipment and other materials. Waste prevention and careful resource use reduce environmental impact and can also lower operating costs. Sustainability works best when it is part of routine purchasing, maintenance and daily decisions.

9.1 Identify and separate waste

Waste should be separated according to local rules and the organisation’s procedures. Common categories include recyclable paper and cardboard, metals, glass, suitable plastics, organic waste and residual waste. Batteries, electronic equipment, lamps and other special waste may need dedicated collection because they can contain hazardous materials.



Figure 9.1. Separate collection bins help people sort recyclable materials. Photograph: Si Griffiths, Wikimedia Commons, CC BY-SA 3.0.

9.2 The waste hierarchy

The preferred order is to prevent waste first, reduce unnecessary use, reuse suitable items, recycle correctly and dispose of residual waste safely. Recycling is valuable, but avoiding unnecessary consumption usually prevents more waste from being created in the first place.

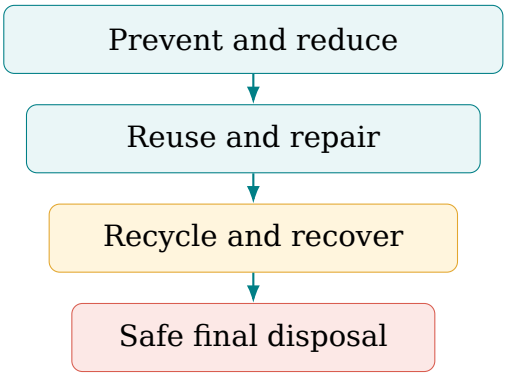


Figure 9.2. A simple waste hierarchy: prevention comes before recycling and disposal.

9.3 Electronic waste

Computers, cables, printers, batteries and other electronic items should not be placed in ordinary waste when local rules require special handling. Use authorised collection or recycling channels, follow data-wiping and asset-disposal policies, and ensure that storage devices containing sensitive information are handled securely.

9.4 Conserving energy and water

Switch off equipment when it is not needed and when policy permits; use power-saving settings; maintain equipment; avoid unnecessary printing; repair leaks; and use water responsibly. Purchasing decisions can consider durability, repairability, energy use and end-of-life disposal, not only the initial price.

9.5 Planning a small waste programme

Start by observing what the workplace throws away and where the waste is produced. Set a realistic target, provide clearly labelled collection points, explain the process, identify an approved collection route and review results periodically. Do not mix separated waste after collection, and never handle unknown or hazardous material without the required training.

Review Questions

1. Put the main waste-management options in preferred order.
2. Why does electronic waste need special attention?
3. List five ways to conserve resources in an office.
4. Outline a simple workplace recycling programme.

Chapter 10

Diversity, Equality and Inclusive Workplaces

An inclusive workplace makes it possible for people with different backgrounds, experiences and abilities to participate and contribute. Respect is shown through fair procedures, accessible communication, equitable opportunities and prompt action when discrimination or harassment occurs.

10.1 Diversity and equality

Diversity describes differences among people, including experience, culture, language, age, gender, disability and perspective. Equality means people are treated fairly and have equal rights and opportunities. Equity recognises that removing barriers may require different forms of support so people can participate meaningfully.

10.2 Everyday inclusive behaviour

Use respectful language, listen without interruption, avoid assumptions about a person's abilities or background, and give credit fairly. Meetings should make room for different voices. Work assignments, training, performance assessment and promotion should follow transparent and relevant criteria.

10.3 Discrimination and harassment

Discrimination involves unfair treatment linked to a protected or personal characteristic, according to applicable law and policy. Harassment can include unwelcome conduct that intimidates, humiliates or creates a hostile environment. Follow organisational reporting channels and relevant law. Bystanders should not escalate a situation; they can support the affected person and report concerns safely.

10.4 Gender equality and work-life responsibilities

Fair workplaces apply consistent standards, provide equal access to learning and advancement, and address barriers that prevent people from participating. Family responsibilities, caregiving and safety concerns should be handled through applicable policies and law rather than stereotypes about who should perform particular roles.

10.5 Building an inclusive team

Use clear selection criteria, accessible training material, fair performance feedback and a reliable process for raising concerns. Review workplace practices regularly. Inclusion is not achieved by a slogan; it requires people to notice barriers and improve the way work is organised.

Review Questions

1. Explain the difference between equality and equity.
2. Give four examples of inclusive behaviour.
3. Why should hiring criteria be transparent?
4. What should a worker do if they experience or witness harassment?

Chapter 11

Accessibility and Policies for Persons with Disabilities

Accessibility means that people can use facilities, information, services and digital tools as independently as possible. A barrier may be physical, communicative, procedural or digital. Inclusive practice identifies barriers and makes reasonable adjustments in line with applicable law and workplace policy.



Figure 11.1. An accessible workplace considers clear space and practical access around desks. Photograph: David290, Wikimedia Commons, CC BY-SA 4.0.

11.1 Recognising different access needs

People may have mobility, vision, hearing, learning, communication or other access requirements. Do not assume that everyone with the same disability needs the same support. Ask respectfully what adjustment is useful, protect privacy and involve the person in decisions that affect their work.

11.2 Digital accessibility

For websites and learning resources, use meaningful headings, descriptive link text, form labels, sufficient colour contrast, keyboard-accessible controls and alternative text for informative images. Do not communicate important information through colour alone. Captions and transcripts can help people who cannot hear audio; text alternatives can help people who cannot access a visual diagram.

11.3 Physical and workplace adjustments

Possible adjustments include accessible routes, suitable desk arrangements, assistive technology, flexible communication formats, adjusted work processes and accessible emergency information. The correct adjustment depends on the person, role, environment and applicable requirements. Keep routes clear and ensure emergency plans include people who may need assistance.

11.4 Respectful support and privacy

Speak directly to the person rather than only to a companion. Ask before offering physical assistance or moving mobility equipment. Do not disclose a person’s disabil-

ity or medical information without an appropriate basis. Apply workplace policies and relevant Indian law, including the Rights of Persons with Disabilities Act, 2016, where applicable; consult official guidance for current obligations.

Review Questions

1. What is an accessibility barrier?
2. Give five examples of digital accessibility practices.
3. Why should adjustments be discussed with the person concerned?
4. How can a workplace improve emergency access for people with disabilities?

Chapter 12

Integrated Case Study and Revision

12.1 Case study: a training-centre portal

A training centre wants a portal to maintain learner details, course enrolments and marks. Staff need to add new learners, update records and generate course summaries. The portal will use PHP to process requests and MySQL to store data. The workplace also needs a safe workstation, inclusive user experience and a responsible plan for retiring old computers.

12.2 Plan the database

Identify entities such as learners, courses and enrolments. Assign primary keys and use foreign keys to represent relationships. Avoid storing repeated course details in every learner row. Decide which fields are required, which values need constraints and which reports are necessary.

12.3 Plan the application

Separate presentation, application logic and database access. Validate input on the server, use prepared statements, encode output and verify permissions for every sensitive action. Do not rely on hidden buttons or client-side checks to protect data. Keep credentials outside public source files and use a dedicated database account.

12.4 Plan quality and workplace responsibility

Write test cases for successful and invalid submissions, access control, database errors and responsive display. Review accessibility with keyboard navigation and readable labels. Organise the work area safely, report hazards, use energy-saving settings and send retired electronic equipment through an approved process.

12.5 Revision questions

1. Explain the role of a DBMS in a web application.
2. Compare primary keys and foreign keys.
3. Write SQL to retrieve students with marks above 70, ordered from highest to lowest.
4. Explain DDL, DML, DCL and transaction-control commands.
5. What is the purpose of SQL aggregate functions?
6. Describe how a PHP request can retrieve data from MySQL.
7. Explain how prepared statements help prevent SQL injection.
8. Compare indexed and associative PHP arrays.
9. What makes a function reusable and easy to test?
10. Define hazard and risk, and explain the hierarchy of controls.
11. Describe a responsible way to manage electronic waste.
12. Give examples of equality, equity and digital accessibility.

12.6 Answer-planning guide

For definition questions, begin with a precise meaning and add one example. For comparison questions, explain the main difference and then compare practical uses. For code questions, state the purpose, show the syntax and describe any important safety condition. For workplace questions, identify the risk or barrier, propose a practical action and mention the relevant procedure or policy. These steps help produce clear, complete answers.